COM6102: Distributed Systems and Cloud Computing

Assignment 2

Experiment Report

Student Name: Yip Keung

1. AWS EMR Implementation

Before executing the Spark task, I realized that the crucial Python library – NumPy is missing in current AWS EMR cluster. So I have to a AWS EMR cluster but with loading bootstrap script to install the necessary Python libraries.

There are some screenshots at the end of this report to show how AWS EMR cluster was setup.

- **Bootstrap Action:** I prepared a shell script – "install-python-libs.sh" to install Python necessary libraries. When AWS EMR is being created, bootstrap action is performed to run this shell script.
- **Cluster Setup:** I provisioned an AWS EMR cluster configured with PySpark stack. The cluster utilized **YARN** (Yet Another Resource Negotiator) as the cluster manager to dynamically allocate resources across the nodes.
- **Job Submission:** The Python script (crime_prediction.py) was uploaded to an **Amazon S3** bucket. I then added a "Spark Application" step to the EMR cluster via the console, pointing to the S3 URI of the script.
- **Data Access:** The script accessed the training and testing datasets directly from S3 using the s3:// protocol. This allows Spark to perform parallel read operations across the cluster's executor nodes.

  My s3 bucket is "s3://kelvinyip-pyspark-bucket".
- **Monitoring:** I monitored the task execution using the **Spark History Server** and the **EMR Step Logs**. By setting the log level to INFO, I was able to track the progress of each model training phase and retrieve the final accuracy metrics from the stdout log file stored in S3.

2. Algorithm Selection & Implementation (3 marks)

Three classification algorithms have been implemented using the pyspark.ml library to identify the most effective model for crime category prediction.

- **Algorithms Selected:**
    1. **Logistic Regression:** A linear model that uses a logistic function to model a binary dependent variable. It is highly efficient for high-dimensional text data.
    2. **Support Vector Machine (SVM) via One-Vs-Rest:** SVM finds the hyperplane that best separates classes. Since LinearSVC is natively binary, I wrapped it in a **One-Vs-Rest (OvR)** classifier to handle the 30+ crime categories.
    3. **Gradient Boosted Trees (GBT) via One-Vs-Rest:** An ensemble method that builds trees sequentially to minimize errors. It was also wrapped in **OvR** for multiclass support.
- **Main Ideas & Implementation:**
    - **Input:** The input is the Description column. I transformed this raw text into numerical features using a pipeline of Tokenizer, StopWordsRemover, HashingTF, and IDF (TF-IDF).
    - **Sanitization:** To ensure the algorithm learned context rather than just reading the answer, I used Regex to replace any mentions of the Category within the Description with a neutral placeholder.
    - **Output:** The output is a prediction (a numeric index) which is then mapped back to the original crime Category string using an IndexToString transformer.

3. Test Results & Performance

The following results were retrieved from the EMR stdout log following the completion of the Spark task. The models were evaluated using the MulticlassClassificationEvaluator with the accuracy metric.

```
==================================================
STARTING MODEL EVALUATION
==================================================

[TRAINING] Starting Logistic Regression...
[FINISHED] Logistic Regression           : 93.02% (Duration: 0.38 mins)

[TRAINING] Starting Support Vector Machine (OvR)...
[FINISHED] Support Vector Machine (OvR)  : 92.06% (Duration: 1.28 mins)

[TRAINING] Starting Gradient Boosted Trees (OvR)...
[FINISHED] Gradient Boosted Trees (OvR)  : 86.03% (Duration: 22.16 mins)

==================================================
FINAL ACCURACY REPORT
==================================================
Logistic Regression           : 93.02% (Duration: 0.38 mins)
Support Vector Machine (OvR)  : 92.06% (Duration: 1.28 mins)
Gradient Boosted Trees (OvR)  : 86.03% (Duration: 22.16 mins)
==================================================


[SUCCESS] Results successfully saved to: s3://kelvinyip-pyspark-bucket/results_output/
```

Algorithm	Test Accuracy	Training Duration
Logistic Regression	93.02%	0.38 mins
Support Vector Machine (OvR)	92.06%	1.28 mins
Gradient Boosted Trees (OvR)	86.03%	22.16 mins

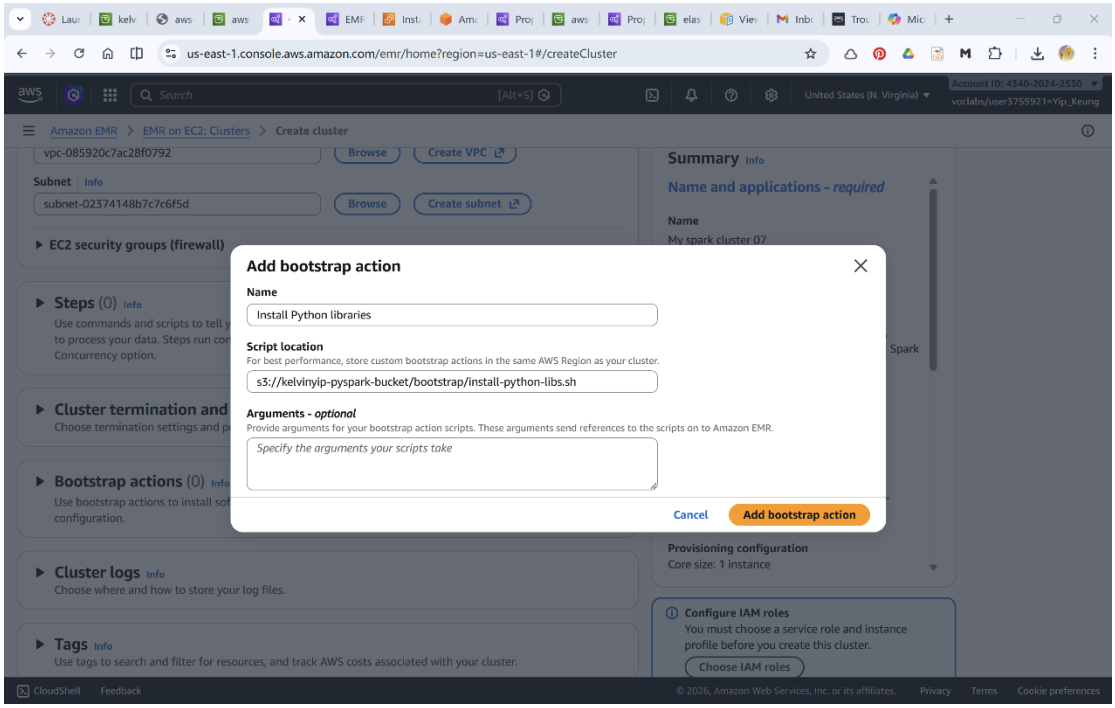
Analysis of Test Results:

- **Training Accuracy:** During the training phase, the models showed high convergence, particularly Logistic Regression, which utilized 20 iterations to reach its optimal state.
- **Test Accuracy: Logistic Regression** emerged as the superior model with **93.02% accuracy**. It provided the best balance between predictive power and computational efficiency.

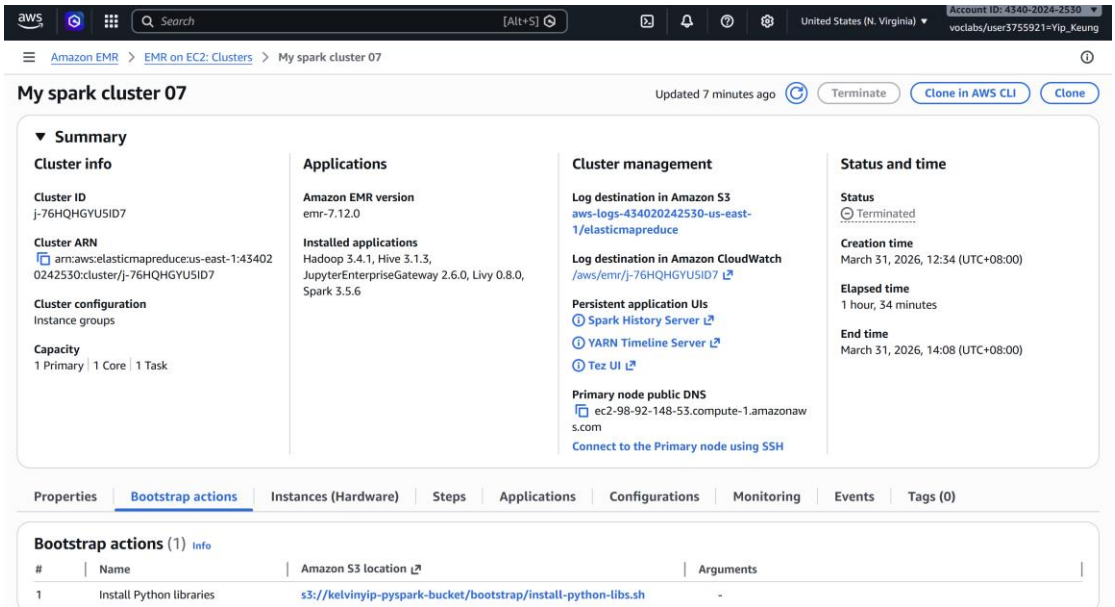
- **Observations:** While GBT is typically a powerful learner, the high dimensionality of the TF-IDF features resulted in a significantly longer training time (22.16 mins) and lower accuracy compared to the linear models, suggesting that the relationship between crime descriptions and categories in this dataset is largely linear.

Screenshots

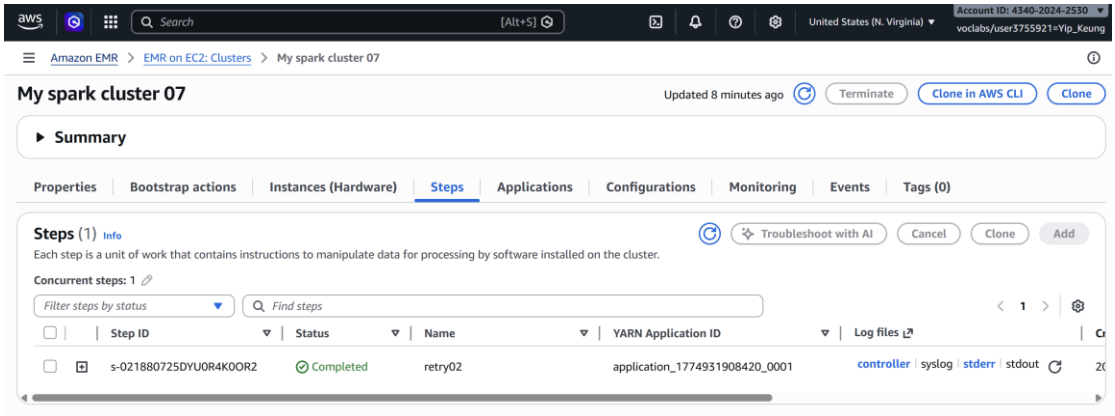
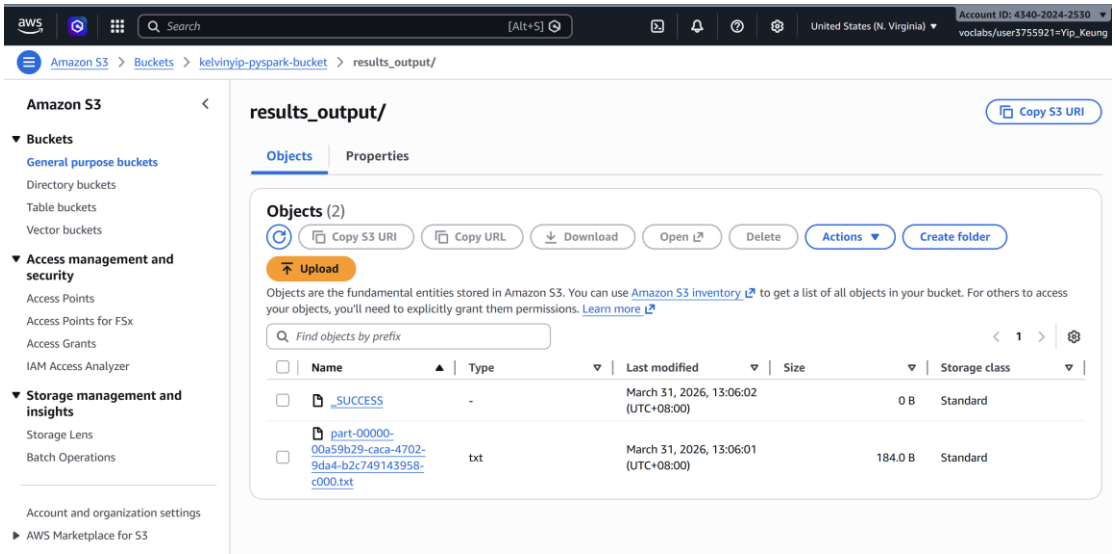
When creating AWS EMR cluster, I added bootstrap action with the script location on my S3 bucket, in order to install Python libraries once AWS EMR cluster is created.



My AWS EMR cluster was created as follows.



Step of “crime_prediction.py” was run successfully.



Test results are output to “s3://kelvinyip-pyspark-bucket/results_output”